Fitting the outcome model

Malcolm Barrett
Stanford University

Outcome Model

```
1 library(broom)
2
3 lm(outcome ~ exposure, data = df, weights = wts) |>
4   tidy()
```

✅ This will get us the point estimate

❌ This will NOT get us the correct confidence intervals

📦 There are three ways to get them right

Three ways to estimate the uncertainty

1. A bootstrap: correct and general purpose, but slow
2. A robust standard error: a sandwich estimator for the outcome model alone
3. A propensity-aware sandwich: correct and fast

The bootstrap is still worth knowing: it also solves problems that have no closed-form solution

1. Create a function to run your analysis once on a sample of your data

```r
fit_ipw <- function(.split, ...) {
  .df <- as.data.frame(.split)

  # fit propensity score model
  propensity_model <- glm(
    net ~ income + health + temperature,
    family = binomial(),
    data = .df
  )

  # calculate inverse probability weights
  .df <- propensity_model |>
    augment(type.predict = "response", data = .df) |>
    mutate(wts = wt_ate(.fitted, net, exposure_type = "binary"))

  # fit correctly bootstrapped ipw model
  lm(malaria_risk ~ net, data = .df, weights = wts) |>
    tidy()
}
```

2. Use {rsample} to bootstrap our causal effect

```
library(rsample)

# fit ipw model to bootstrapped samples
bootstrapped_net_data <- bootstraps(
  net_data,
  times = 1000,
  apparent = TRUE
)

bootstrapped_net_data
```

2. Use {rsample} to bootstrap our causal effect

```
# Bootstrap sampling with apparent sample
# A tibble: 1,001 × 2
   splits             id
   <list>             <chr>
 1 <split [1752/640]> Bootstrap0001
 2 <split [1752/638]> Bootstrap0002
 3 <split [1752/637]> Bootstrap0003
 4 <split [1752/635]> Bootstrap0004
 5 <split [1752/639]> Bootstrap0005
 6 <split [1752/616]> Bootstrap0006
 7 <split [1752/652]> Bootstrap0007
 8 <split [1752/638]> Bootstrap0008
 9 <split [1752/629]> Bootstrap0009
```

2. Use {rsample} to bootstrap our causal effect

```
fit_ipw(bootstrapped_net_data$splits[[1]])
```

```
# A tibble: 2 × 5
  term        estimate std.error statistic  p.value
  <chr>          <dbl>     <dbl>     <dbl>    <dbl>
1 (Intercept)     43.4     0.470      92.3 0
2 netTRUE        -12.3     0.663     -18.5 8.96e-70
```

2. Use {rsample} to bootstrap our causal effect

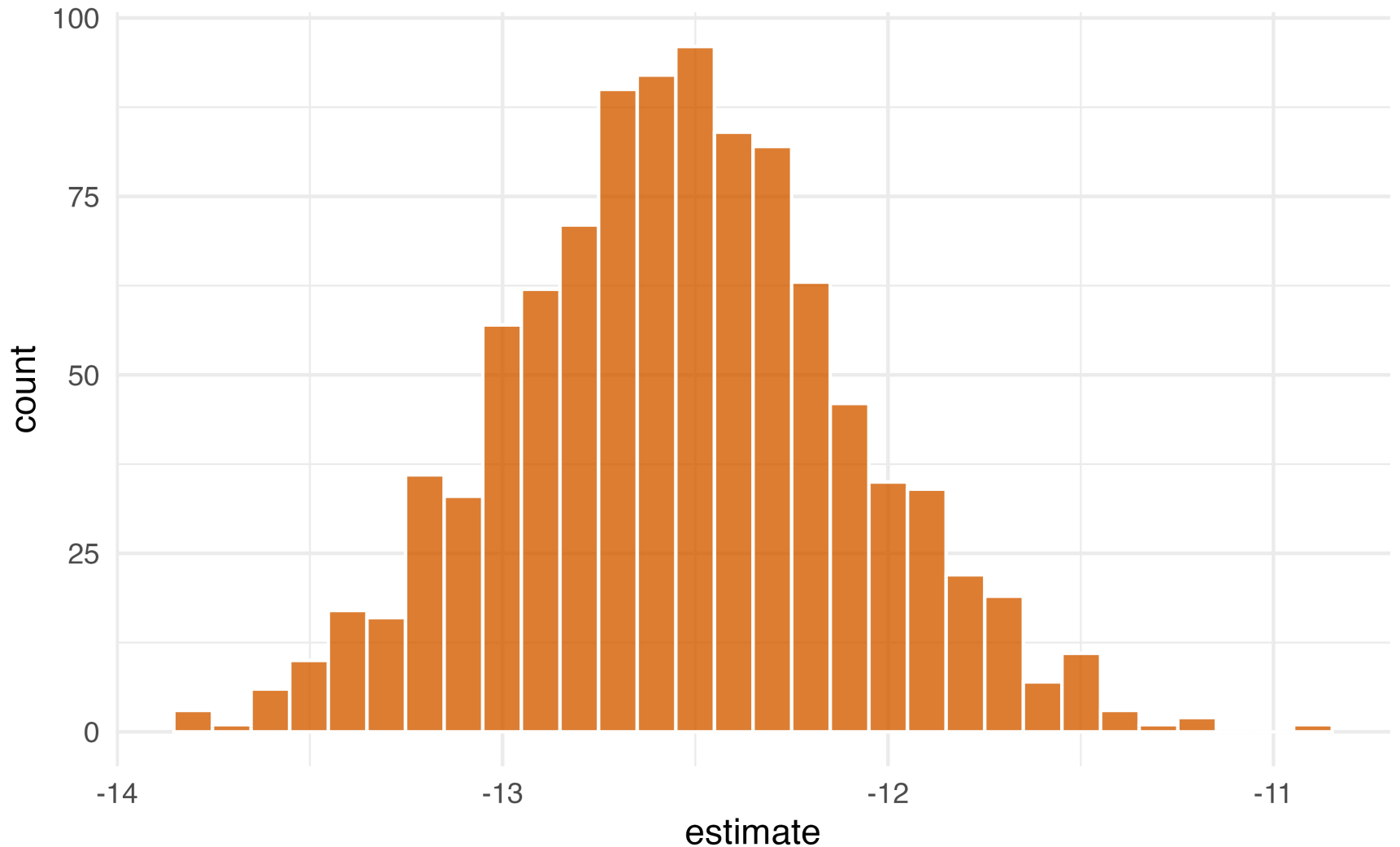
```
ipw_results <- bootstrapped_net_data |>
  mutate(boot_fits = map(splits, fit_ipw))

ipw_results
```

2. Use {rsample} to bootstrap our causal effect

```
# Bootstrap sampling with apparent sample
# A tibble: 1,001 × 3
   splits             id            boot_fits
   <list>             <chr>         <list>
 1 <split [1752/640]> Bootstrap0001 <tibble [2 × 5]>
 2 <split [1752/638]> Bootstrap0002 <tibble [2 × 5]>
 3 <split [1752/637]> Bootstrap0003 <tibble [2 × 5]>
 4 <split [1752/635]> Bootstrap0004 <tibble [2 × 5]>
 5 <split [1752/639]> Bootstrap0005 <tibble [2 × 5]>
 6 <split [1752/616]> Bootstrap0006 <tibble [2 × 5]>
 7 <split [1752/652]> Bootstrap0007 <tibble [2 × 5]>
 8 <split [1752/638]> Bootstrap0008 <tibble [2 × 5]>
 9 <split [1752/629]> Bootstrap0009 <tibble [2 × 5]>
```

2. Use {rsample} to bootstrap our causal effect



3. Pull out the causal effect

```r
# get t-statistic-based CIs
boot_estimate <- int_t(ipw_results, boot_fits) |>
  filter(term == "netTRUE")

boot_estimate
```

```
# A tibble: 1 × 6
  term    .lower .estimate .upper .alpha .method
  <chr>    <dbl>     <dbl>  <dbl>  <dbl> <chr>
1 netTRUE  -13.4     -12.5  -11.6   0.05 student-t
```

Fit both models on the full data

```r
propensity_model <- glm(
  net ~ income + health + temperature,
  family = binomial(),
  data = net_data
)

net_data_wts <- propensity_model |>
  augment(type.predict = "response", data = net_data) |>
  mutate(wts = wt_ate(.fitted, net, exposure_type = "binary"))

ipw_model <- lm(malaria_risk ~ net, data = net_data_wts, weights = wts)
```

Robust standard errors are not enough

```
robust_se <- avg_comparisons(ipw_model, variables = "net", vcov = "HC3")

bind_rows(
  "lm() default" = tidy(ipw_model) |> filter(term == "netTRUE"),
  "HC3 robust SE" = tidy(robust_se),
  "ipw()" = tidy(ipw(propensity_model, ipw_model)),
  .id = "method"
) |>
  select(method, estimate, std.error)
```

```
# A tibble: 3 × 3
  method        estimate std.error
  <chr>            <dbl>     <dbl>
1 lm() default     -12.5     0.624
2 HC3 robust SE    -12.5     0.758
3 ipw()            -12.5     0.437
```

❌ Robust SEs account for the weights, but not for estimating them

ipw(): correct and fast

- `ipw()` uses a sandwich estimator that also accounts for the uncertainty in the propensity score model

- Bring your own models: you fit the propensity score and outcome models yourself, then supply them to `ipw()`

```
1 ipw_fit <- ipw(propensity_model, ipw_model)
```

ipw(): correct and fast

```
1 ipw_fit
```

```
Inverse Probability Weight Estimator
Estimand: ATE
Effects: marginal (population-averaged)

Weight Estimator:
  Call: glm(formula = net ~ income + health + temperature, family = binomial(),
    data = net_data)

Outcome Model:
  Call: lm(formula = malaria_risk ~ net, data = net_data_wts, weights = wts)

Marginal estimates:
      estimate   std.err       z ci.lower ci.upper conf.level   p.value
diff -12.54476   0.43712 -28.698  -13.402  -11.688       0.95 < 2.2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

`ipw()` reports the estimand along with the estimate

The bootstrap and ipw() agree

```
# A tibble: 2 × 4
  method    estimate conf.low conf.high
  <chr>        <dbl>    <dbl>     <dbl>
1 bootstrap    -12.5    -13.4     -11.6
2 ipw()        -12.5    -13.4     -11.7
```

`ipw()` fits each model once and takes a fraction of a second; the bootstrap fits 1,001 pairs of models

Your Turn

Create a function called `fit_ipw` that fits the propensity score model and the weighted outcome model for the effect between `park_extra_magic_morning` and `wait_minutes_posted_avg`

Using the `bootstraps()` and `int_t()` functions to estimate the final effect.

Fitting both models on the full data and using `ipw()` to compare the two intervals.

Binary outcomes: an odds ratio

```r
net_data_wts <- net_data_wts |>
  mutate(risk_over_50 = malaria_risk > 50)

glm(
  risk_over_50 ~ net,
  data = net_data_wts,
  weights = wts,
  # quasibinomial() avoids a warning about non-integer successes
  family = quasibinomial()
) |>
  tidy(exponentiate = TRUE, conf.int = TRUE) |>
  filter(term == "netTRUE") |>
  select(estimate, conf.low, conf.high)
```

```
# A tibble: 1 × 3
  estimate conf.low conf.high
     <dbl>    <dbl>     <dbl>
1    0.225    0.169     0.296
```

Binary outcomes: a risk difference

```
lm(risk_over_50 ~ net, data = net_data_wts, weights = wts) |>
  tidy(conf.int = TRUE) |>
  filter(term == "netTRUE") |>
  select(estimate, conf.low, conf.high)
```

```
# A tibble: 1 × 3
  estimate conf.low conf.high
     <dbl>    <dbl>     <dbl>
1   -0.205   -0.240    -0.169
```

The default standard errors still do not account for the propensity score model; the same bootstrap and sandwich approaches apply here